Team 05

IPv6 Hate Club

White Team Documentation

ICDC 2026 Cyber Defense Competition

Central Downtown City Transportation Security

February 21, 2026

Table of Contents

Documentation Methodology *4*

Security Audit Procedures *5*

System-Wide Security Audits **5**
Access Control Audits 5
Binary and Library Analysis 5
Network Reconnaissance 5

Backdoor Detection and Removal **5**
User Account Audits 5
Persistence Mechanism Checks 5
Application Security Audits 6

System Hardening Measures **6**
Attack Prevention 6
File System Security 6
Password and Authentication 6
Credential Security 7

Network Security Implementation **7**

Vulnerability Remediation Summary *8*

Public Services *11*

Network Architecture *12*

Network Segmentation Strategy **12**

INTERNAL Network Rules **12**

MANAGEMENT Network Rules **13**

PUBLIC Network Rules **13**

WAN Configuration and External Connectivity **13**

Application Security *15*

CSRF via State-Mutating GET Endpoints (Critical — CWE-352) **15**

Hardcoded SECRET_KEY in Backup File (Critical — CWE-798) **15**

Path Traversal via Ticket IDs (Critical — CWE-22) **15**

Werkzeug Debug Mode Enabled (High — CWE-489) **16**

Backend Service Running as Root (High — CWE-250) **16**

Additional Application Vulnerabilities Remediated **16**

Threat Hunting Activities *18*

Windows Systems (AD, Tickets, Admin) **18**
Scheduled Tasks 18
WMI Event Subscriptions 18
Registry Run Keys and Services 18

Backdoor User Accounts 18
Linux Systems (WWW, Mgmt) **18**
Malicious Systemd Services 18
LD_PRELOAD Rootkit 19
SUID Binary Backdoors 19
Backdoor User Accounts 19
PAM and Cron Audits 19
System Hardening ***20***
Windows Systems **20**
SMB Hardening (All Windows Systems) 20
Print Spooler Disabled (AD, Admin) 20
Credential Security (AD, Admin) 20
Active Directory Hardening 20
WinRM and Service Hardening (Admin) 20
Windows Defender 20
Linux Systems **21**
PwnKit Remediation (Mgmt) 21
Service Minimization 21
SSH Configuration 21
Application Configuration 21
Current System Status ***22***
Challenges and Resolutions ***23***
OPNsense Gateway Configuration **23**
Virtual IP Requirement **23**
Proxy Configuration **23**
Backend Connectivity Timeout **23**
PwnKit Patching on AlmaLinux 8.4 ..... **24**
Ongoing Security Measures ***25***
Continuous Monitoring **25**
Incident Response Procedures **25**
User Access Management **25**
Conclusion ***26***

Documentation Methodology

We utilized Notion as our central documentation and task management platform throughout the competition preparation and hardening phase. This allowed our eight-person team to categorize all vulnerabilities discovered, track remediation progress, and document all changes made to systems in real time. Each team member could access and update the shared workspace, ensuring comprehensive documentation of our security operations.

The Notion workspace was organized into the following sections:

- **Vulnerability Tracking:** Each discovered vulnerability was logged with the affected system, severity level, description, and remediation status.
- **Task Assignment:** Security tasks were assigned to specific team members with priority levels to ensure parallel work across all five systems.
- **Change Log:** All system modifications were documented with timestamps, affected systems, and configuration details.
- **Network Diagrams:** Visual representations of network architecture, OPNsense interface assignments, and firewall rule logic.
- **Credentials Vault:** Secure storage of updated passwords and access credentials, shared only among team members.

This systematic approach ensured that no vulnerability went unaddressed and that all team members had visibility into the overall security posture of our infrastructure.

Security Audit Procedures

We conducted systematic security audits across all systems following a comprehensive checklist documented in our Notion workspace. The following procedures were completed to identify vulnerabilities, remove backdoors, and harden our infrastructure against Red Team attacks.

System-Wide Security Audits

Access Control Audits

- Audited sudoers configuration and wheel group membership on all Linux systems (WWW, Mgmt)
- Verified Administrator group membership and removed unauthorized accounts on all Windows systems (AD, Tickets, Admin)
- Confirmed account permissions align with scenario role requirements
- Disabled guest accounts on all Windows systems

Binary and Library Analysis

- Audited all SUID and SGID binaries on Linux systems for unauthorized executables
- Checked for LD_PRELOAD hijacking and malicious shared libraries across all Linux systems
- Verified integrity of critical system binaries (sudo, su, ssh, sshd, passwd)
- Removed four malicious SUID backdoor binaries from WWW (resetpwd, ush, ush2, ticketmon)

Network Reconnaissance

- Verified all listening services on every system using ss -tulpn (Linux) and netstat -ano (Windows)
- Closed unnecessary ports and services on all systems
- Confirmed only required services are externally accessible per scenario requirements
- Documented all listening services and their purposes

Backdoor Detection and Removal

User Account Audits

- Removed backdoor user account 'taco' (UID 1001) from WWW server
- Removed unauthorized local accounts 'nsa' and 'scrat' with admin and RDP privileges from Admin server
- Verified no accounts have blank passwords on any system
- Disabled guest accounts on all Windows systems (AD, Tickets, Admin)

Persistence Mechanism Checks

- Discovered and removed netcat reverse shell backdoor (ticket_mon.service) with three simultaneous listeners on ports 1337, 12069, and 12345 on WWW

- Discovered and neutralized LD_PRELOAD rootkit (nsabackdoor.so) with ghost file injection and command execution capabilities on WWW
- Removed SUID root password reset backdoor (/usr/bin/resetpwd) that silently reset root password to 'cdc' on WWW
- Removed two SUID root shell backdoors (/usr/bin/ush and /usr/bin/ush2) on WWW
- Audited scheduled tasks, WMI event subscriptions, and registry Run keys on all Windows systems
- Audited cron jobs, systemd timers, and at jobs on all Linux systems
- Confirmed no reverse shells present on any machine

Application Security Audits

- Performed full source-code review of the Flask frontend on WWW (Python modules, Jinja2 templates, JavaScript files)
- Audited MGMT backend Flask application for authentication bypass, debug mode, and credential exposure
- Reviewed Tickets application for JWT signing vulnerabilities relevant to ticket forgery flag
- Audited Admin UI for XSS vulnerabilities, LDAP authentication implementation, and hardcoded credentials

System Hardening Measures

Attack Prevention

- Disabled SMBv1 on all Windows systems to prevent EternalBlue exploitation
- Disabled SMBv3 compression on all Windows systems to prevent SMBGhost exploitation
- Enforced SMB signing on all Windows systems to prevent relay attacks
- Disabled Print Spooler service on all Windows systems to prevent PrintNightmare exploitation
- Disabled WDigest cleartext credential storage on AD and Admin servers

File System Security

- Fixed world-writable JavaScript files (map.js, tickets.js) on WWW to 644 permissions
- Verified proper permissions on /etc/shadow (000) on all Linux systems
- Secured Flask secret key file (.secret_key) with 600 permissions on WWW
- Placed blue flag files with correct permissions (644 for /etc/ on Linux, System32 on Windows)

Password and Authentication

- Changed all default passwords (root, cdc, Administrator) on every system
- Updated MariaDB credentials and application configuration on Mgmt
- Implemented password strength validation (minimum 8 characters) on WWW frontend registration

- Disabled reversible encryption on all AD user accounts

Credential Security

- Disabled WDigest cleartext password storage on AD and Admin servers
- Enabled LSA protected mode (RunAsPPL) on AD to prevent credential dumping
- Verified Zerologon (CVE-2020-1472) patched and secure channel enforcement enabled on AD
- Enforced LDAP signing via Group Policy on AD to prevent relay attacks

Network Security Implementation

- Implemented three-tier network segmentation (INTERNAL, MANAGEMENT, PUBLIC) using separate Proxmox bridges through OPNsense
- Configured thirteen NAT port forward rules to map external IPs to internal systems
- Created five Virtual IPs on the WAN interface for all service external addresses
- Configured firewall rules on WAN, INTERNAL, MANAGEMENT, and PUBLIC interfaces with default deny
- Blocked all cross-segment pivot paths (PUBLIC to INTERNAL, PUBLIC to MANAGEMENT, MANAGEMENT to INTERNAL)

This comprehensive audit approach ensured that we systematically addressed all potential security vulnerabilities and persistence mechanisms across our entire infrastructure. Every item was verified and documented in our Notion workspace with assigned team members and completion status.

Vulnerability Remediation Summary

The following table provides a comprehensive summary of all vulnerabilities discovered and remediated during our security hardening phase. All items have been fully addressed and verified.

Vulnerability	System	Description
LD_PRELOAD Rootkit (nsabackdoor.so)	WWW	Neutralized LD_PRELOAD rootkit with ghost file injection and command execution capabilities. Removed binary, persistence script, and cleared environment — RESOLVED
Netcat Reverse Shell (ticket_mon.service)	WWW	Removed systemd service launching three simultaneous netcat listeners on ports 1337, 12069, and 12345 — RESOLVED
SUID Password Reset Backdoor	WWW	Removed /usr/bin/resetpwd that silently reset root password to 'cdc' — RESOLVED
SUID Root Shell Backdoors	WWW	Removed /usr/bin/ush (compiled ELF) and /usr/bin/ush2 (script), both with SUID root permissions — RESOLVED
Backdoor User Account 'taco'	WWW	Removed unauthorized local user account (UID 1001) created for persistent SSH access — RESOLVED
Insecure Flask Session Configuration	WWW	Implemented persistent SECRET_KEY with secure file storage, enabled HttpOnly and SameSite cookie attributes — RESOLVED
CSRF via State-Mutating GET Endpoints	WWW	Converted four routes (/activate_ticket, /scan_ticket, /delete_ticket, /buy_ticket) from GET to POST with CSRF token validation — RESOLVED
Hardcoded SECRET_KEY in Backup File	WWW	Deleted config.py.backup containing SECRET_KEY = 'cdc' and HTTPONLY = False — RESOLVED
Path Traversal via Ticket IDs	WWW	Added regex validation for all ticket IDs before URL construction to prevent backend path traversal — RESOLVED
Unauthenticated /api/vehicles Endpoint	WWW	Added @login_required decorator to vehicle data proxy endpoint — RESOLVED
World-Writable JavaScript Files	WWW	Fixed map.js and tickets.js from 777 to 644 permissions — RESOLVED
Missing Authentication on Routes	WWW	Added @login_required decorator to all protected routes (profile, payment, scan, password) — RESOLVED
Rate Limiting Not Implemented	WWW	Implemented three-layer rate limiting with flask-limiter: per-endpoint strict limits, global defaults, and 429 error handler — RESOLVED
HTML Injection in Payment Data	WWW	Added html.escape() to all fields in payment_tools.py sanitize function — RESOLVED
CVE-2021-4034 PwnKit	Mgmt	Upgraded polkit from vulnerable 0.115-11.el8 to patched 0.115-15.el8_10.2 — RESOLVED

Backend Service Running as Root	Mgmt	Changed backend.service from User=root to User=cdc to limit blast radius — RESOLVED
Werkzeug Debug Mode Enabled	Mgmt	Removed WERKZEUG_DEBUG_PIN from /etc/backend.env and unused DebuggedApplication import — RESOLVED
atd Job Scheduler Active	Mgmt	Disabled atd service to prevent scheduled backdoor persistence — RESOLVED
System User with Login Shell	Mgmt	Changed pcpqa user shell from /bin/bash to /sbin/nologin — RESOLVED
Hardcoded DB/LDAP Credentials	Mgmt	Updated MariaDB password and LDAP bind credentials in config.py — RESOLVED
LDAP Auth Bypass in Code	Mgmt	Verified broken auth bypass (ldap_exists) was commented out and real LDAP authentication active — RESOLVED
WDigest Cleartext Credentials	AD	Set UseLogonCredential = 0 to disable cleartext password storage in memory — RESOLVED
Reversible Encryption on Accounts	AD	Disabled AllowReversiblePasswordEncryption on all user accounts and domain policy — RESOLVED
SMBv1 Enabled (EternalBlue)	AD	Disabled SMBv1 protocol and enabled SMB signing — RESOLVED
Print Spooler (PrintNightmare)	AD	Stopped and disabled Print Spooler service — RESOLVED
LDAP Signing Not Enforced	AD	Enforced LDAP signing via Group Policy for both server and client — RESOLVED
LSA Not Protected	AD	Enabled RunAsPPL to prevent credential dumping (Mimikatz-style attacks) — RESOLVED
ADCS ESC1/ESC2 Templates	AD	Locked down certificate templates, enabled manager approval, tightened enrollment permissions — RESOLVED
PowerShell Logging Disabled	AD	Enabled module logging and script block logging for detection and forensics — RESOLVED
SMBv1 Enabled (EternalBlue)	Tickets	Disabled SMBv1 protocol feature — RESOLVED
SMBv3 Compression (SMBGhost)	Tickets	Applied DisableCompression = 1 registry value — RESOLVED
SMB Signing Not Enforced	Tickets	Enabled RequireSecuritySignature on server and workstation — RESOLVED
Rogue Account 'nsa'	Admin	Disabled unauthorized local admin account with RDP access, planted Jan 16, 2026 — RESOLVED
Rogue Account 'scrat'	Admin	Disabled unauthorized local admin account with RDP access, planted Jan 13, 2026 — RESOLVED
Guest Account Enabled	Admin	Disabled guest account to prevent unauthenticated access — RESOLVED
SMBv3 Compression (SMBGhost)	Admin	Applied DisableCompression = 1 registry value — RESOLVED

Print Spooler Running	Admin	Stopped and disabled Print Spooler service — RESOLVED
WinRM Exposed	Admin	Stopped and disabled WinRM service, blocked port 5985 via firewall rule — RESOLVED
WDigest Not Configured	Admin	Set UseLogonCredential = 0 to disable cleartext credential storage — RESOLVED
Windows Update Disabled	Admin	Re-enabled wuauserv and set to Automatic startup — RESOLVED
Default Passwords (cdc)	All Systems	Changed all default passwords on every system and updated application credentials — RESOLVED
Terminal History Exposure	All Systems	Cleared all terminal and bash history files — RESOLVED

Public Services

We have configured the following services to be publicly accessible for IScorE scoring:

Domain Name	External IP	Internal IP	Ports / Services
ad.team5.isucdc.com	64.5.53.10	10.10.10.10	389 (LDAP), 636 (LDAPS), 3389 (RDP)
tickets.team5.isucdc.com	64.5.53.20	10.10.20.10	443 (HTTPS), 5000 (HTTP API), 3389 (RDP)
mgmt.team5.isucdc.com	64.5.53.30	10.10.20.30	22 (SSH), 5000 (HTTP API)
admin.team5.isucdc.com	64.5.53.40	10.10.20.20	80 (HTTP), 3389 (RDP)
www.team5.isucdc.com	64.5.53.50	10.10.30.10	22 (SSH), 80 (HTTP)

Network Architecture

We implemented a defense-in-depth network architecture using three isolated network segments controlled by an OPNsense firewall. Due to competition infrastructure limitations, we utilized separate Proxmox bridges instead of VLAN tagging to achieve Layer 2 network isolation.

Network Segmentation Strategy

Our network design segregates systems based on trust level and function:

- **INTERNAL Network (10.10.10.0/24):** Critical infrastructure containing only the Active Directory domain controller. This is the most protected segment with the strictest inbound restrictions.
- **MANAGEMENT Network (10.10.20.0/24):** Backend services including the Tickets application server, Admin UI, and Management backend API.
- **PUBLIC Network (10.10.30.0/24):** Internet-facing systems containing only the WWW public website. Highest exposure to external attacks.

This architecture enforces security boundaries through OPNsense firewall rules that control all inter-segment traffic. Layer 2 isolation is achieved using separate Proxmox bridges (team050, team051, team052), which prevents systems on different bridges from communicating without routing through the firewall.

Segment	Bridge	Subnet	Systems	Risk Level
INTERNAL	team050	10.10.10.0/24	AD (10.10.10.10)	HIGHEST SECURITY
MANAGEMENT	team051	10.10.20.0/24	Tickets (.10), Admin (.20), Mgmt (.30)	HIGH
PUBLIC	team052	10.10.30.0/24	WWW (10.10.30.10)	HIGHEST EXPOSURE

INTERNAL Network Rules

The INTERNAL network contains our most critical infrastructure — the Active Directory domain controller. We configured the following firewall policy:

- INTERNAL systems have full outbound access for management operations and software updates
- Other networks can access AD only on ports 389 and 636 for LDAP/LDAPS authentication
- All other inbound connections to the INTERNAL segment are blocked
- All connections are logged for security monitoring and incident response

We placed the Active Directory server in this isolated segment because it is the authentication backbone for all domain-joined systems. Compromise of AD would allow an attacker to pivot to all machines, so we isolated it behind strict firewall rules. All systems must remain domain-joined to AD to maintain eligibility for competition placement.

MANAGEMENT Network Rules

The MANAGEMENT network hosts backend services and the ticketing application. We configured the following firewall policy:

- MANAGEMENT systems can access AD on ports 389 and 636 for LDAP/LDAPS domain authentication
- MANAGEMENT systems have outbound internet access via the competition proxy (199.100.16.100:3128)
- MANAGEMENT systems cannot initiate connections to the INTERNAL network beyond LDAP
- A REJECT rule with logging blocks any pivot attempts from MANAGEMENT to INTERNAL

This design prevents lateral movement if a backend service is compromised. An attacker gaining access to the Tickets or Admin server would be blocked from reaching the Active Directory server directly.

PUBLIC Network Rules

The PUBLIC network contains the internet-facing WWW server. This is our most exposed segment because a public web server is the most likely initial attack vector. We configured the following firewall policy:

- WWW can reach Mgmt on port 5000 (backend API calls required for frontend functionality)
- WWW can reach AD on port 389 for LDAP authentication
- PUBLIC systems have outbound proxy access (199.100.16.100:3128)
- **PUBLIC to INTERNAL: BLOCKED** with logging
- **PUBLIC to MANAGEMENT: BLOCKED** with logging (except the specific WWW to Mgmt:5000 rule above)

The WWW to Mgmt:5000 opening is necessary because the frontend Flask application depends on the backend API for login, ticket operations, and vehicle data. Without this rule the website is non-functional. All other cross-segment traffic from the PUBLIC network is denied. If Red Team compromises the public web server, they cannot pivot to the Active Directory server or any other backend service beyond the single allowed API port.

WAN Configuration and External Connectivity

Our WAN interface presented unique challenges due to the competition network design. We received /32 host routes instead of a traditional subnet, which required special configuration:

- **Far Gateway Configuration:** The gateway (64.5.53.254) is not on the same Layer 2 segment as our WAN IPs. We enabled the 'Far Gateway' and 'Upstream Gateway' options in OPNsense to allow routing to a gateway outside our local subnet.
- **Virtual IPs:** All five service external IPs (64.5.53.10 through .50) were added as IP Alias Virtual IPs on the WAN interface, each with a /32 mask. Without these, OPNsense

silently discards traffic destined for addresses it does not own. The firewall's own WAN address is 64.5.53.70.

- **NAT Port Forwards:** Thirteen port forward rules map external IPs and ports to internal systems, enabling external access to LDAP, LDAPS, RDP, SSH, HTTPS, and HTTP services across all five systems plus firewall management.
- **Proxy Configuration:** All internet-bound traffic routes through the competition proxy (199.100.16.100:3128). We configured this on individual systems via /etc/profile (Linux) and Settings > Proxy (Windows).

Application Security

We identified and remediated critical vulnerabilities in the Flask frontend application (WWW), the Flask backend application (Mgmt), and reviewed the Tickets and Admin UI applications. Our security audit revealed multiple high-severity issues that could allow attackers to compromise the transit system infrastructure.

CSRF via State-Mutating GET Endpoints (Critical — CWE-352)

Location: WWW — app/payment.py, four routes

Problem: Four routes (/activate_ticket, /scan_ticket, /delete_ticket, /buy_ticket) used GET requests but performed destructive or purchasing actions. Because Flask-WTF CSRF protection only covers POST/PUT/DELETE/PATCH, any logged-in user visiting a malicious page could have had their tickets deleted or a purchase triggered silently via an embedded <img> tag.

Fix Applied: All four routes were converted from GET to POST. JavaScript fetch calls were refactored to POST with X-CSRFToken headers sourced from a new <meta name="csrf-token"> tag added to the base layout. The buy and scan forms were converted to method="post" with hidden CSRF token inputs.

Verification: Confirmed all four endpoints reject GET requests and require valid CSRF tokens on POST. Tested with missing and invalid tokens to confirm 400/403 responses.

Hardcoded SECRET_KEY in Backup File (Critical — CWE-798)

Location: WWW — app/config.py.backup

Problem: A backup configuration file contained SECRET_KEY = 'cdc' (trivially guessable) and SESSION_COOKIE_HTTPONLY = False. An attacker could use a tool like flask-unsign to forge a session cookie for any user, including administrators, using the known key.

Fix Applied: The backup file was deleted from disk entirely. The live configuration was verified to use a randomly generated 64-character hex secret key persisted securely to a file with 600 permissions.

Verification: Confirmed config.py.backup no longer exists. Verified the live secret key file is 64 characters, owned by cdc, with permissions 600.

Path Traversal via Ticket IDs (Critical — CWE-22)

Location: WWW — app/payment.py, all ticket routes

Problem: All ticket routes concatenated raw user input directly onto backend URLs (e.g., /tickets/activate/ + str(id)). An input like ../admin or 1%2f..%2fsecret could reach unintended backend endpoints, potentially accessing or modifying data outside the ticket scope.

Fix Applied: Added a strict regex validator (_SAFE_ID_RE = re.compile(r'^[a-zA-Z0-9_-]{1,64}$')) and a _get_id() helper function that validates every ticket ID before it touches a URL. Invalid IDs return a 400 response immediately.

Verification: Tested with path traversal payloads including ../, %2f, and URL-encoded variants. All rejected with 400 status. Legitimate ticket IDs continue to function correctly.

Werkzeug Debug Mode Enabled (High — CWE-489)

Location: Mgmt — /etc/backend.env, app/__init__.py

Problem: The backend environment file contained WERKZEUG_DEBUG_PIN='123456789' and the application factory imported DebuggedApplication. Werkzeug debug mode provides an interactive Python console to anyone who can trigger an exception, enabling arbitrary code execution on the server.

Fix Applied: Removed WERKZEUG_DEBUG_PIN from /etc/backend.env and removed the unused DebuggedApplication import from __init__.py. Confirmed debug mode is OFF in service logs.

Verification: Triggered intentional errors and confirmed no debug console is presented. Service logs confirm 'Debug mode: off'.

Backend Service Running as Root (High — CWE-250)

Location: Mgmt — /etc/systemd/system/backend.service

Problem: The backend Flask application ran as root (User=root in the systemd unit). Any vulnerability in the application — such as the debug console or a code injection — would grant an attacker immediate root-level access to the entire system.

Fix Applied: Changed the service to run as the cdc user (User=cdc), limiting the blast radius of any application-level compromise.

Verification: Confirmed backend.service runs as cdc user via systemctl show. Application continues to function correctly with reduced privileges.

Additional Application Vulnerabilities Remediated

Beyond the critical findings above, our code review identified and fixed numerous additional issues across the frontend and backend applications:

Vulnerability	System	Remediation
Unauthenticated /api/vehicles endpoint	WWW	Added @login_required to prevent unauthenticated backend proxy access
Missing @login_required on protected routes	WWW	Added authentication checks to profile, payment, scan, and password change routes
HTML injection in payment display	WWW	Added html.escape() to all fields in sanitize_payment_info function

SECRET_KEY regenerated on every restart	WWW	Implemented persistent key storage to .secret_key file with 600 permissions
Config loaded after blueprint registration	WWW	Moved app.config.from_object(Config) to immediately after app creation
Missing /scan route	WWW	Added Flask route for scan.html which existed but was unreachable
Incorrect login redirect in decorator	WWW	Fixed url_for to redirect to GET login page instead of POST-only handler
No timeouts on backend HTTP calls	WWW	Added timeout=10 to all requests.get/post calls to prevent hanging
No password strength validation	WWW	Added _validate_password() enforcing minimum 8 characters on register and change-password
Stale form data in registration	WWW	Fixed to use cleaned/stripped username variable instead of raw form input
CVE-2021-4034 PwnKit privilege escalation	Mgmt	Upgraded polkit to patched version 0.115-15.el8_10.2
atd job scheduler enabled	Mgmt	Disabled atd service to prevent scheduled backdoor persistence
System user pcpqa with login shell	Mgmt	Changed shell from /bin/bash to /sbin/nologin
Hardcoded DB and LDAP credentials	Mgmt	Updated MariaDB password and LDAP bind password in config.py

Threat Hunting Activities

We conducted systematic threat hunting across all systems to identify and remove Red Team persistence mechanisms. Our approach involved checking multiple layers where attackers commonly establish backdoors.

Windows Systems (AD, Tickets, Admin)

Scheduled Tasks

We enumerated all scheduled tasks on each Windows system and filtered for non-Microsoft entries. On the Tickets machine, only the legitimate 'Tickets' scheduled task (application auto-start) was present. On the Admin machine, only the legitimate 'Admin UI' task and Edge updater were found. On AD, all scheduled tasks were verified as legitimate Microsoft entries. No malicious scheduled tasks were detected on any Windows system.

WMI Event Subscriptions

We queried the root\Subscription namespace for __EventFilter, __EventConsumer, and __FilterToConsumerBinding objects on all Windows systems. On AD, all WMI subscriptions were confirmed clean. On the Admin machine, only the built-in SCM Event Log Filter/Consumer pair was found, which is standard Windows infrastructure. No malicious WMI persistence mechanisms were detected.

Registry Run Keys and Services

We audited HKLM and HKCU Run/RunOnce keys on all Windows systems. On the Admin machine, only SecurityHealth (Windows Defender) and AzureArcSetup entries were found, both legitimate. We also audited all Windows services for suspicious entries, checking binary paths and service names. No malicious registry entries or services were detected.

Backdoor User Accounts

We discovered two rogue local accounts on the Admin machine: 'nsa' (planted January 16, 2026) and 'scrat' (planted January 13, 2026, comment: '1337 h4x0r'). Both had been added to the local Administrators group with RDP access. We disabled both accounts. The Guest account was also found enabled on Admin and was disabled. On AD, the backdoor user 'taco' was found in Active Directory on WWW (documented separately). No unauthorized accounts were found on the Tickets machine.

Linux Systems (WWW, Mgmt)

Malicious Systemd Services

On WWW, we discovered ticket_mon.service, a systemd service disguised with the description 'TicketServ-rs' to blend in with legitimate ticketing infrastructure. It launched three simultaneous netcat listeners piping input directly to bash, providing unauthenticated root shell access on ports 1337, 12069, and 12345. The service was stopped, disabled, and all associated files

(/usr/bin/ticketmon, the unit file) were removed. We verified no processes remained listening on those ports after cleanup.

LD_PRELOAD Rootkit

On WWW, we discovered a sophisticated LD_PRELOAD rootkit (nsabackdoor.so) persisted through /etc/profile.d/setup.sh, which injected the rootkit into every user process on login. Analysis of the rootkit source code (/usr/src/debug/nsabackdoor.c) revealed three attack capabilities: ghost file injection (fake 'PLATYPUS' files appearing in directory listings as a detection canary), filesystem operation interception (hooked stat/lstat/statx calls returning fake metadata), and a command execution backdoor triggered by a reversed string pattern ('roodkcabasn$$$') in any read() call. The rootkit was quarantined, the persistence mechanism removed, and environment variables cleared. No LD_PRELOAD issues were found on Mgmt.

SUID Binary Backdoors

On WWW, we discovered four malicious SUID binaries planted by Red Team between January 7–10, 2026: /usr/bin/resetpwd (a script that reset the root password to 'cdc'), /usr/bin/ush (a compiled ELF binary providing a root shell), /usr/bin/ush2 (a script-based root shell fallback), and /usr/bin/ticketmon (the netcat payload). All four were removed. A full SUID audit of the remaining 25 binaries confirmed all are legitimate system utilities. On Mgmt, no malicious SUID binaries were found.

Backdoor User Accounts

On WWW, user 'taco' (UID 1001) was found as a local account with /bin/bash shell and a SHA-512 password hash. The account showed no evidence of active use (no SSH logins, no files owned by UID 1001 outside the home directory), suggesting it was a fallback persistence mechanism. The account was removed with userdel -r and no orphaned files remained. On Mgmt, no unauthorized user accounts were found.

PAM and Cron Audits

PAM configuration was audited on both Linux systems. No malicious pam_exec, pam_script, pam_permit, or credential-logging modules were found in /etc/pam.d/ on either system. Cron jobs were audited across /etc/crontab, /etc/cron.d/, /etc/cron.hourly/ through cron.weekly/, and all user crontabs. No malicious entries were found on either system. At jobs were checked and confirmed empty. On Mgmt, the atd service was additionally disabled as a preventive measure.

System Hardening

Beyond removing Red Team persistence mechanisms, we implemented defensive hardening measures across all systems to reduce the attack surface and protect against known exploits.

Windows Systems

SMB Hardening (All Windows Systems)

We applied comprehensive SMB hardening across all three Windows systems (AD, Tickets, Admin) to protect against EternalBlue (CVE-2017-0144), SMBGhost (CVE-2020-0796), and relay attacks. SMBv1 was disabled on all systems. SMBv3 compression was disabled via the DisableCompression registry value. SMB signing was enforced on both server and workstation components by setting RequireSecuritySignature = 1 on LanmanServer and LanmanWorkstation.

Print Spooler Disabled (AD, Admin)

The Print Spooler service was stopped and set to Disabled startup type on the AD domain controller and Admin machine to eliminate the PrintNightmare (CVE-2021-34527) attack surface. This service is not required for the competition scenario.

Credential Security (AD, Admin)

WDigest cleartext password storage was disabled on both the AD domain controller and Admin machine by setting UseLogonCredential = 0. On AD, LSA protected mode (RunAsPPL = 1) was enabled to prevent credential dumping tools like Mimikatz from accessing LSASS memory. Reversible encryption was audited and disabled on all AD user accounts. Credential Guard configuration was reviewed and enabled where supported.

Active Directory Hardening

Zerologon (CVE-2020-1472) was verified patched with FullSecureChannelProtection = 1 confirmed in the Netlogon parameters. No Event ID 5829 entries were found in the System log. LDAP signing was enforced via Group Policy on both the server and client sides. Active Directory Certificate Services templates were audited and locked down against ESC1 and ESC2 privilege escalation paths — enrollment flags were reviewed, manager approval was enabled on vulnerable templates, and overly broad enrollment permissions were tightened. Ticketing machine certificate enrollment was verified still functional. PowerShell module logging and script block logging were enabled for detection and forensics capabilities.

WinRM and Service Hardening (Admin)

On the Admin machine, WinRM was found running and exposing port 5985. The service was stopped, disabled, and a Windows Firewall rule was created to block port 5985 inbound. Windows Update (wuauserv) was found stopped with Manual startup type and was re-enabled with Automatic startup to ensure critical security patches are installed.

Windows Defender

Windows Defender was verified installed and active across all Windows systems (AD, Tickets, Admin). Real-time protection, anti-malware service, and antivirus components were all confirmed enabled.

Linux Systems

PwnKit Remediation (Mgmt)

The Mgmt server was running polkit version 0.115-11.el8, which is vulnerable to CVE-2021-4034 (PwnKit) local privilege escalation. This was patched by upgrading to version 0.115-15.el8_10.2. The WWW server running AlmaLinux 9 was already patched (polkit 0.117-14.el9 ships with the fix). The Dirty Cow vulnerability (CVE-2016-5195) was confirmed patched on WWW as the kernel version (5.14.0) includes the fix.

Service Minimization

On Mgmt, the atd job scheduler was disabled to prevent Red Team from scheduling persistent backdoors. The system user pcpqa had a login shell (/bin/bash) changed to /sbin/nologin. On WWW, the backend service was verified running as expected and unnecessary Performance Co-Pilot services (pmcd, pmlogger) were identified as localhost-only and not externally reachable. The codeserver.service (VSCode Server) was confirmed running as the cdc user and bound only to localhost for team use during the competition.

SSH Configuration

SSH configuration was reviewed on both Linux systems. PermitRootLogin yes was found in /etc/ssh/sshd_config.d/01-permitrootlogin.conf on WWW (generated by the Anaconda installer). SSH authorized keys were verified clean on both systems — no unauthorized keys were found in any user or root .ssh/authorized_keys files. Shadow file permissions were verified as 000 (root-only access) on both systems.

Application Configuration

On WWW, the Flask frontend was hardened with a persistent SECRET_KEY (stored in .secret_key with 600 permissions), CSRF protection via flask-wtf, HttpOnly and SameSite=Lax session cookies, and three-layer rate limiting. On Mgmt, the Flask backend was demoted from root to cdc user, Werkzeug debug mode was fully disabled, and MariaDB was confirmed bound to localhost only (127.0.0.1:3306). Hardcoded credentials in the Mgmt application configuration were updated with strong values.

Current System Status

All required services are operational and accessible for IScorE scoring. All systems are domain-joined to ad.team5.isucdc.com. The following table summarizes the current status of each system:

System	Status	Services	Notes
AD	Operational	LDAP (389), RDP (3389)	Zerologon patched, LDAP signing enforced, ADCS locked down
Tickets	Operational	HTTPS (443), RDP (3389)	SMB hardened, application running and compiling correctly
Mgmt	Operational	SSH (22), API (5000)	PwnKit patched, backend running as cdc user, debug mode off
Admin	Operational	HTTP (80), RDP (3389)	Rogue accounts disabled, WinRM blocked, Defender active
WWW	Operational	SSH (22), HTTP (80)	Rootkit eliminated, all app vulns fixed, blue flag placed
OPNsense	Operational	Firewall, NAT, Routing	Far Gateway configured, 3-tier segmentation active

Challenges and Resolutions

During the setup and hardening phase, we encountered several technical challenges that required creative solutions.

OPNsense Gateway Configuration

Challenge: The gateway (64.5.53.254) was not on the same Layer 2 segment as our WAN interface IP addresses. All WAN IPs were configured as /32 host routes, and attempting to add a default route resulted in 'Network is unreachable' errors.

Root Cause: Standard gateway configuration requires the gateway to be on the local subnet. With /32 routes, OPNsense could not reach the gateway at Layer 2.

Solution: We discovered the 'Far Gateway' option under System → Gateways → Single → Edit WAN_GW. Enabling both 'Far Gateway' and 'Upstream Gateway' options allowed OPNsense to successfully route traffic through the competition gateway at 64.5.53.254.

Virtual IP Requirement

Challenge: NAT port forward rules were configured correctly but external traffic was being silently dropped. Services appeared unreachable from outside despite correct internal configurations.

Root Cause: OPNsense only accepts traffic for IPs it explicitly owns. With five external IPs but only one primary WAN address, the firewall was discarding traffic for the other four.

Solution: All five service external IPs (64.5.53.10 through .50) plus the firewall IP (64.5.53.70) needed to be added as IP Alias Virtual IPs on the WAN interface before creating NAT rules. Once the Virtual IPs were in place, all thirteen port forward rules functioned correctly.

Proxy Configuration

Challenge: All systems needed to use the competition proxy (199.100.16.100:3128) for internet access. Windows systems did not have this configured by default, Linux package managers needed specific proxy settings, and applications required individual configuration.

Solutions: On Linux systems, proxy variables were configured in /etc/profile for shell sessions. On Windows systems, proxy was set in Settings → Network & Internet → Proxy. Package managers (dnf on AlmaLinux, NuGet on Windows) were individually configured to route through the proxy.

Backend Connectivity Timeout

Challenge: The WWW frontend could not connect to the Mgmt backend at mgmt.team5.isucdc.com:5000, causing login functionality and all API-dependent features to fail with connection timeout errors.

Root Cause: The OPNsense PUBLIC interface firewall rules were not yet configured to allow WWW (10.10.30.10) to reach Mgmt (10.10.20.30) on port 5000 across segments.

Solution: A specific PASS rule was added to the PUBLIC interface allowing WWW to reach Mgmt:5000. Once applied, the frontend successfully connected to the backend and all functionality was restored.

PwnKit Patching on AlmaLinux 8.4

Challenge: The Mgmt server needed a polkit upgrade to patch CVE-2021-4034, but the outdated AlmaLinux 8.4 release package caused GPG key mismatches when trying to install from the repositories.

Solution: The --nogpgcheck flag was used for the specific polkit upgrade (dnf update polkit --nogpgcheck -y), and the patched version (0.115-15.el8_10.2) was verified installed via rpm -q polkit.

Ongoing Security Measures

We continue to maintain and improve our security posture through ongoing monitoring and hardening activities.

Continuous Monitoring

- Regular review of OPNsense firewall logs for blocked connection attempts and lateral movement patterns
- Monitoring systemd service status across all Linux systems for unauthorized service creation
- Checking Windows scheduled tasks, WMI subscriptions, and registry Run keys for new persistence mechanisms
- Reviewing Active Directory user accounts and group memberships for unauthorized changes
- Monitoring network connections (ss -tnp, netstat) for suspicious outbound connections or reverse shells
- Periodic SUID binary audits on Linux systems to detect newly planted backdoors

Incident Response Procedures

- Check OPNsense and system logs to identify the attack vector
- Isolate the affected system if necessary while maintaining scoring service availability
- Remove persistence mechanisms (services, cron jobs, user accounts, SUID binaries)
- Change compromised credentials immediately and update application configurations
- Patch the exploited vulnerability
- Document all findings and remediation steps for White Team reporting
- Verify services are restored and scoring checks pass

User Access Management

All systems are joined to the Active Directory domain (ad.team5.isucdc.com) for centralized user management. Default credentials have been changed across all systems. Domain password policy has been configured with reversible encryption disabled. Blue flag files have been placed in their correct locations with appropriate permissions on all five systems.

Conclusion

We have successfully implemented a defense-in-depth security architecture for Central Downtown City's public transportation infrastructure. Our three-tier network segmentation strategy isolates critical infrastructure from public-facing systems, particularly protecting the Active Directory server from the PUBLIC network where the internet-facing web server operates.

We identified and neutralized a sophisticated LD_PRELOAD rootkit with ghost file injection and command execution capabilities, along with multiple SUID backdoors, a netcat reverse shell service, and rogue user accounts planted across the environment. On the Windows side, we removed backdoor accounts, hardened SMB against EternalBlue, SMBGhost, and relay attacks, disabled vulnerable services like Print Spooler and WinRM, and secured credential storage.

We conducted a comprehensive source-code review of both the Flask frontend and backend applications, identifying and remediating critical vulnerabilities including CSRF on state-mutating endpoints, path traversal via ticket IDs, exposed backup files with hardcoded secrets, an enabled Werkzeug debug console, unauthenticated endpoints, and a privilege escalation via a root-running backend service. We implemented rate limiting, CSRF protection, input validation, and proper authentication across all application routes.

We hardened Active Directory against Zerologon, credential dumping, LDAP relay attacks, and certificate services abuse. We patched PwnKit on the management server, enforced least-privilege service accounts, and disabled unnecessary services across the environment. All five systems are domain-joined, all required services are operational for IScorE scoring, and blue flags are placed with correct permissions.

Through systematic documentation in our Notion workspace and methodical vulnerability remediation across our eight-person team, we achieved comprehensive hardening across all systems. We continue to monitor for new Red Team activity and maintain our security posture through ongoing threat hunting and system hardening activities.